B5 Data validation

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B5 Data validation 3
B5-L1 DataFrame schemas with pandera 3
B5-L2 pydantic models for config and input records 7
B5-L3 Validate at the boundaries, with severity 12
Track B best-practice checklist 18
Appendix: solutions and answer keys 19

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B5 Data validation

Roadmap: Prerequisite for all months. Schema checks with pandera and input models with pydantic.

B5-L1 DataFrame schemas with pandera

Concept

A **schema** states what a DataFrame must look like: which columns exist, their dtypes, and rules on their values. **pandera** checks a pandas DataFrame against a schema and raises when it does not match. Examples here use pandera 0.33 with pandas 3.

```python
import pandera.pandas as pa

RESPONSES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "age": pa.Column(int, pa.Check.in_range(16, 99)),
        "weight": pa.Column(float, pa.Check.gt(0)),
    },
    strict=True,
    coerce=True,
)
clean = RESPONSES.validate(raw, lazy=True)
```

- `pa.Column(dtype, checks, nullable=False)` is the default: nulls fail unless you set `nullable=True`.
- `unique=True` rejects duplicate keys.
- `strict=True` rejects columns the schema does not name. `coerce=True` converts dtypes before checking.
- `lazy=True` collects **every** failure into one `SchemaErrors` exception. Its `failure_cases` attribute is a DataFrame with the column, check and failing value. Without `lazy`, validation stops at the first `SchemaError`.

`SchemaErrors` is not a subclass of `SchemaError`. Catch both when you handle failures. A `strict=True` violation raises `SchemaErrors` even without `lazy=True`.

The class syntax, `pa.DataFrameModel`, does the same with type hints. Add `@pa.check_types` to a function whose parameters are annotated `DataFrame[Model]`, and pandera validates inputs and outputs on every call.

Import from `pandera.pandas`. The plain `import pandera` still works in 0.33, but the pandas-specific module is the documented path.

pandera does not run in this app's browser runtime. The exercises check your code's structure. Run `pytest` locally with the `dev` group to execute schemas for real.

Why it matters

A supplier changes a weight export so that non-responders get weight `0` instead of being left out. The weighting script divides by the sum of weights per cell. Three cells now contain only zeros. The script writes `NaN` shares to the client dashboard, and nobody sees an error.

`pa.Column(float, pa.Check.gt(0))` on the input fails the run on day one, with the failing rows listed in `failure_cases`.

Worked example: weak code

```
1  def load_weights(path):
2      df = pd.read_csv(path)
1      assert "weight" in df.columns
2      df = df.dropna()
5      return df
```

1. **Line 3.** Checks one column exists. Not its type, its values or other columns. And python -O strips asserts.
2. **Line 4.** Silently drops rows with any null, in any column. The count of dropped rows is never reported.

Worked example: strong code

```
1  import pandas as pd
2  import pandera.pandas as pa
3
4  WEIGHTS = pa.DataFrameSchema(
5      {
1          "respondent_id": pa.Column(str, unique=True),
7          "cell": pa.Column(str),
2          "weight": pa.Column(float, pa.Check.gt(0)),
9      },
3      strict=True,
11     coerce=True,
12 )
13
14
15 def load_weights(path: str) -> pd.DataFrame:
16     """Read and validate a weights file. Grain: one row per respondent.
17
18     Raises:
19         pa.errors.SchemaErrors: Listing every failing row and check.
20     """
```

```
❹     return WEIGHTS.validate(pd.read_csv(path, dtype={"respondent_id":
    "str"}), lazy=True)
```

1. **Line 6.** The grain, enforced: one row per respondent.
2. **Line 8.** Not null (the default) and positive: zero weights fail loudly.
3. **Line 10.** A new or renamed column in the supplier file fails the run.
4. **Line 21.** lazy=True reports every failure at once, not just the first.

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

Recap card

- **pandera Column defaults** nullable=False. Set nullable=True where nulls are expected.
- **strict=True and coerce=True** strict rejects unknown columns. coerce converts dtypes before checks.
- **validate(df, lazy=True)** Runs every check and raises SchemaErrors with failure_cases.
- **SchemaError vs SchemaErrors** Separate classes. SchemaErrors is not a subclass. Catch both, or better, let them raise.

Practice

Exercise 1. Multiple choice (level 1)

In pandera 0.33, what does `lazy=True` change in `schema.validate(df, lazy=True)`?

- a) Validation runs later, when the DataFrame is first used.
- b) Every failure is collected and raised together as SchemaErrors.
- c) Failing rows are dropped instead of raising.
- d) Only dtypes are checked, not values.

Why?

- r1) You see all problems in one run, with failure_cases listing each one.
- r2) It is faster because it skips checks.

Exercise 2. Predict the output (level 2)

What does this print? (pandera 0.33)

```
1 import pandas as pd
2 import pandera.pandas as pa
3
4 schema = pa.DataFrameSchema({"w": pa.Column(float, pa.Check.gt(0))})
5 try:
```

```
    schema.validate(pd.DataFrame({"w": [1.0, -2.0, -3.0]}), lazy=True)
except pa.errors.SchemaErrors as err:
    print(err.failure_cases["failure_case"].tolist())
```

Exercise 3. Fill the gap (level 2)

Reject duplicate IDs, and reject any column the schema does not name.

```
schema = pa.DataFrameSchema(
    {"respondent_id": pa.Column(str, _____(1)=True), "score":
    pa.Column(int)},
    _____(2)=True,
)
```

Exercise 4. Write the function (level 3)

Define a module constant `SCORES` as a pandera `DataFrameSchema` with:

- `respondent_id`: str, unique.
- `nps`: int, between 0 and 10 inclusive.
- `comment`: str, nulls allowed.
- Unknown columns rejected, dtypes coerced.

Then write `validate_scores(df: pd.DataFrame) -> pd.DataFrame` that returns `SCORES.validate(df, lazy=True)`.

pandera does not run in the browser, so the checker reads your code. Run it locally with `pytest` to see it validate.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SCORES = ...

# Validate an NPS extract. Grain: one row per respondent.
def validate_scores(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd
import pandera.pandas as pa


def validate_scores(df):
    return df
```

Exercise 5. Spot the bug (level 4)

A bad weights file passed through this loader and broke the dashboard. Tap the line that let it through.

```
import pandas as pd
import pandera.pandas as pa
```

```
3
4  WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
       pa.Check.gt(0))}, strict=True)
5
6
7  def load_weights(path: str) -> pd.DataFrame:
8      """Read and validate weights."""
9      df = pd.read_csv(path)
10     try:
11         return WEIGHTS.validate(df, lazy=True)
12     except pa.errors.SchemaError:
13         return df
```

Write down the line number of the bug. Then write the fix.

Sources

- pandera 0.33: lazy=True raises SchemaErrors with a failure_cases DataFrame. (verified: pandera documentation; run on pandera 0.33.1 with pandas 3.0.6 in this repository.)
- SchemaErrors is not a subclass of SchemaError in pandera 0.33. (verified: issubclass check on pandera 0.33.1 in this repository.)
- A strict=True violation raises SchemaErrors even without lazy=True. (verified: Run on pandera 0.33.1 in this repository.)
- import pandera.pandas as pa is the documented import for pandas schemas. (search-only: pandera documentation (docs site not reachable from this environment).)

B5-L2 pydantic models for config and input records

Concept

A dataclass (lesson B2-L2) does not check types. **pydantic** does. A `BaseModel` validates and converts data when you build it, and raises one `ValidationError` that lists every problem. Examples use pydantic 2.12.

```
1  from pydantic import BaseModel, ConfigDict, Field
2
3
4  class WeightingConfig(BaseModel):
5      model_config = ConfigDict(extra="forbid", frozen=True)
6
7      weight_col: str
8      min_base: int = Field(default=50, gt=0)
9      cap: float = Field(default=5.0, gt=1.0)
10
11
```

```
cfg = WeightingConfig.model_validate(yaml.safe_load(text))
```

- `Field(gt=0, le=10)` adds bounds. `@field_validator("name")` adds your own rule.
- `extra="forbid"` rejects unknown keys, so `minbase: 30` in a YAML file fails instead of being ignored. The default, `"ignore"`, drops them silently.
- `frozen=True` rejects assignment after construction.
- `e.errors()` returns a list of dicts with `loc`, `type` and `msg`. Log it as structured data.

pydantic runs in **lax mode** by default. It converts `"30"` to `30` and `30.0` to `30`, but rejects `"thirty"` and `30.5`. It also converts `True` to `1` for an `int` field. Set `ConfigDict(strict=True)` when inputs must already have the right type.

One gap: `model_copy(update={...})` does **not** validate. `cfg.model_copy(update={"min_base": -1})` returns a config with `min_base = -1`. To change a validated model, build a new one: `WeightingConfig.model_validate({**cfg.model_dump(), "min_base": 30})`.

Use pydantic for records and settings that arrive from outside: YAML, JSON, API payloads. Use pandera (lesson B5-L1) for whole DataFrames. For a list of records, `TypeAdapter(list[Model]).validate_python(rows)` validates them in one call.

Why it matters

A weighting config file had `cap: 0.5` typed where `5` was meant, and `minbase` spelt without the underscore. The loader used a plain dict. The cap shrank every weight above 0.5 down to 0.5, which flattened the weighting, and `min_base` stayed at its default. The report went out with unweighted-looking results.

`WeightingConfig.model_validate` with `gt=1.0` and `extra="forbid"` rejects both mistakes in one error message, before any data is read.

Worked example: weak code

```
1 def load_config(raw):
2     return {
3         "weight_col": raw.get("weight_col", "weight"),
4         "min_base": int(raw.get("min_base", 50)),
5         "cap": float(raw.get("cap", 5.0)),
6     }
```

1. **Line 3.** A missing key falls back to a guess.
2. **Line 4.** A typo such as minbase is ignored. int() accepts negatives.
3. **Line 5.** No bounds: 0.5 passes.

Worked example: strong code

```
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    """Settings for one weighting run, loaded from YAML."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


def load_config(raw: dict[str, object]) -> WeightingConfig:
    """Validate a parsed YAML mapping.

    Raises:
        pydantic.ValidationError: Listing every invalid or unknown key.
    """
    return WeightingConfig.model_validate(raw)
```

1. **Line 7.** Unknown keys fail. No changes after loading.
2. **Line 9.** Required: no default, so a missing key fails.
3. **Line 11.** A bound that encodes what a cap means.

Validate external settings and records once, at the boundary, with every rule in the model.

Recap card

- **pydantic extra keys, default behaviour** Ignored silently. Use ConfigDict(extra="forbid") for config.
- **pydantic lax mode** Converts "30" and 30.0 to 30, True to 1. Use strict=True to refuse conversions.
- **model_copy(update=...)** Does not validate. Rebuild with model_validate({**m.model_dump(), ...}).
- **pydantic or pandera** pydantic for records and settings from outside. pandera for whole DataFrames.

Practice

Exercise 1. Multiple choice (level 1)

A pydantic 2 model has the default config. A YAML file adds an unknown key `minbase: 30`. What happens on `model_validate`?

- a) ValidationError: extra_forbidden

- b) The key is ignored silently.
- c) min_base is set to 30.
- d) A warning is printed.

Why?

- r1) The default extra setting is "ignore". Set extra="forbid" to reject unknown keys.
- r2) pydantic never checks keys.

Exercise 2. Predict the output (level 2)

What does this print? (pydantic 2.12)

```python
from pydantic import BaseModel, Field


class Cfg(BaseModel):
    min_base: int = Field(default=50, gt=0)


cfg = Cfg(min_base="30")
print(cfg.model_copy(update={"min_base": -1}).min_base)
```

Exercise 3. Fill the gap (level 2)

Reject unknown keys, stop changes after loading, and require `nps` between 0 and 10.

```python
class Response(BaseModel):
    model_config = ConfigDict(extra="_____(1)", _____(2)=True)

    respondent_id: str
    nps: int = Field(ge=0, _____(3)=10)
```

Exercise 4. Write the function (level 3)

Write a pydantic model `WeightingConfig` and a function `load_config(raw)`.

- Fields: `weight_col: str` (required), `min_base: int` default 50 and greater than 0, `cap: float` default 5.0 and greater than 1.0.
- Reject unknown keys. Reject assignment after construction.
- `load_config(raw: dict[str, object]) -> WeightingConfig` validates the mapping.

Your code must pass these checks:

- A valid mapping must build a config with defaults filled in.
- cap must be greater than 1.0 and min_base greater than 0.
- Unknown keys such as minbase must fail. Set extra="forbid".

- weight_col is required. Give it no default.
- Assignment after construction must fail. Set frozen=True.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Settings for one weighting run.
class WeightingConfig(BaseModel): ...

# Validate a parsed YAML or JSON mapping.
def load_config(raw: dict[str, object]) -> WeightingConfig: ...
```

Starter code:

```python
from pydantic import BaseModel, ConfigDict, Field


def load_config(raw):
    return raw
```

Exercise 5. Spot the bug (level 4)

A job lowers the minimum base for a small market. The run then reports subgroups of size 0 as valid. Tap the line with the bug and fix the function so the new value is validated.

```python
from pydantic import BaseModel, ConfigDict, Field


class Cfg(BaseModel):
    """Run settings."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    min_base: int = Field(default=50, gt=0)


def with_min_base(cfg: Cfg, value: int) -> Cfg:
    """Return a copy of cfg with a new min_base."""
    return cfg.model_copy(update={"min_base": value})
```

Write down the line number of the bug. Then write the fix.

Sources

- pydantic 2 default extra behaviour is ignore; extra="forbid" raises extra_forbidden. (verified: pydantic documentation; run on pydantic 2.12.5 and 2.13.5 in this repository.)
- Lax mode converts "30" and 30.0 to int 30 and True to 1, and rejects 30.5 and "thirty". (verified: pydantic conversion table; run on pydantic 2.12.5 in this repository.)

- model_copy(update=...) does not validate the update. (verified: pydantic documentation for model_copy; run on pydantic 2.12.5 in this repository.)

B5-L3 Validate at the boundaries, with severity

Concept

Validation belongs at **boundaries**, where data crosses from one owner to another:

- **On the way in.** After reading a file, API response or config. Check schema, keys and value ranges (lessons B5-L1 and B5-L2).
- **On the way out.** Before writing a table or sending a report. Check the promises you make to the next consumer: grain, row counts, totals, shares that sum to 1.

Checks in the middle of a pure transformation add noise. A function that receives validated input and is unit tested does not need to re-check it.

Not every failed check should stop a run. Give each check a **severity**:

- **CRITICAL** stops the run. Wrong grain, missing required column, empty input, weights that do not sum to the sample size.
- **WARN** records the problem and continues. Missing share above its usual level, a subgroup below its reporting base.

Record results as **structured data**, not prose, so a dashboard or alert filters them:

```python
Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    check: str
    severity: Severity
    passed: bool
    observed: float | None
```

Run all checks, collect the results, log them, **then** raise if any CRITICAL check failed. The log keeps every result, even on the run that stops.

Two coding rules make checks trustworthy:

- **Guard every division.** An empty input has no missing share. Return `None`, not a `ZeroDivisionError`.
- **Test `None` with `is None`.** `if share:` treats a real share of `0.0` like a missing one, and the check silently never runs.

Why it matters

A tracker QA step printed "WARNING: low base in 3 cells" to the notebook output and carried on. Nobody reads notebook output for scheduled jobs. The same step never checked for an empty input. The week a supplier delivered an empty file, the job computed shares from zero rows, divided by zero, and wrote `NaN` to every cell of the client dashboard.

Structured results in a table get alerts. A CRITICAL `min_rows` check stops the run before the empty file reaches the dashboard.

Worked example: weak code

```
def qa(df):
    share = df["weight"].isna().sum() / len(df)
    if share:
        if share > 0.1:
            print("WARNING: many missing weights")
    if len(df) == 0:
        print("empty file")
```

1. **Line 2.** Divides by zero on an empty file, before the emptiness check runs.
2. **Line 3.** Truthiness: a share of 0.0 skips the check.
3. **Line 5.** Prose on stdout. Nothing stores or alerts on it.
4. **Line 7.** An empty file is critical, but the run carries on.

Worked example: strong code

```
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome, stored in the run log."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) ->
   list[QaResult]:
    """Evaluate every input check. Never raises."""
    n = len(df)
```

```
    share = None if n == 0 else float(df["weight"].isna().sum()) / n
    return [
        QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
        QaResult("missing_weight", "WARN", share is not None and share <=
max_missing, share),
    ]


def enforce(results: list[QaResult]) -> None:
    """Raise when any CRITICAL check failed."""
    failed = [r.check for r in results if r.severity == "CRITICAL" and
not r.passed]
    if failed:
        raise RuntimeError(f"critical checks failed: {failed}")
```

1. **Line 6.** A closed set of severities, checked by mypy.
2. **Line 22.** Guarded division: no share on an empty frame.
3. **Line 25.** is None, so a real 0.0 passes the check instead of skipping it.
4. **Line 33.** Raised after all checks ran, so the log has the full picture.

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

Recap card

- **Where to validate** At boundaries: after reading input, before writing output.
- **CRITICAL vs WARN** CRITICAL stops the run (raise). WARN records and continues.
- **Order of QA operations** Run every check, log all results, then raise if any CRITICAL failed.
- **Two coding rules for checks** Guard every division. Test for None with is None, never truthiness.

Practice

Exercise 1. Multiple choice (level 1)

A pipeline reads a supplier file, runs five pure transforms, then writes a client table. Where do validation checks belong?

- a) After every transform.
- b) After reading, and before writing.
- c) Only before writing.
- d) Nowhere, if the transforms have unit tests.

Why?

- r1) Those are the boundaries where data changes owner. Tests cover the pure middle.
- r2) Fewer checks run faster.

Exercise 2. Predict the output (level 2)

No weights are missing this week. What does this print?

```python
def missing_check(share: float | None, limit: float) -> str:
    if share:
        return "pass" if share <= limit else "fail"
    return "no data"


print(missing_check(0.0, 0.1))
```

Exercise 3. Put in order (level 2)

Order the steps of one pipeline run.

- Run input checks and collect results
- Write the client table
- Run output checks on the result
- Run the pure transforms
- Log all results, then raise if any CRITICAL failed
- Read the supplier file

Exercise 4. Write the function (level 3)

Write `run_checks(df, min_rows, max_missing) -> list[QaResult]` and `enforce(results) -> None`. `QaResult` and `Severity` are given.

- `min_rows`, CRITICAL: passes when the row count is at least `min_rows`. `observed` is the row count as a float.
- `missing_weight`, WARN: the share of null `weight` values. Passes when the share is at most `max_missing`. On an empty frame the share is `None` and the check fails. `observed` is the share.
- `run_checks` never raises. `enforce` raises `RuntimeError` naming each failed CRITICAL check, and ignores WARN failures.

Your code must pass these checks:

- Check names, severities, observed values and pass flags must match the brief.
- A missing share of 0.0 must pass, with observed 0.0.

- An empty frame must fail min_rows, and give a None share without ZeroDivisionError.
- enforce must raise RuntimeError naming the failed CRITICAL check.
- A WARN failure alone must not raise.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One check outcome.
class QaResult: ...

# Evaluate input checks. Never raises.
def run_checks(
    df: pd.DataFrame,
    min_rows: int,
    max_missing: float,
) -> list[QaResult]: ...

# Stop the run when any CRITICAL check failed.
def enforce(results: list[QaResult]) -> None: ...
```

Starter code:

```python
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df, min_rows, max_missing):
    ...


def enforce(results):
    ...
```

Exercise 5. Architecture design (level 4)

Assign a severity to each check on a weekly brand-tracking run.

Options: CRITICAL, WARN.

1. respondent_id is unique in the input
2. A brand's subgroup base is below 50
3. Weights sum to the sample size within 0.1%
4. Share of missing age is above last month's level

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B5-L1 DataFrame schemas with pandera

B5-L1 exercise 1

Hints. (1) Without lazy, when does validation stop? (2) What does the exception hold with lazy? (3) failure_cases, one row per failure.

Answer. b) Every failure is collected and raised together as SchemaErrors. Because: r1) You see all problems in one run, with failure_cases listing each one.

Without `lazy`, the first failing check raises `SchemaError`. With it, pandera runs every check and raises `SchemaErrors` with all failures.

Common wrong answers.

- `a`: Validation runs at once. lazy refers to collecting errors, not delaying them.
- `c`: pandera raises. Dropping rows needs an explicit choice, not a flag on validate.
- `d`: All checks run. lazy changes how failures are reported.
- `b|r2`: Right answer, wrong reason. lazy runs every check, so it is not faster.

B5-L1 exercise 2

Hints. (1) Which values fail gt(0)? (2) Does lazy stop at the first? (3) All failing values are listed.

Answer.

```
1 [-2.0, -3.0]
```

Both negative weights fail. `lazy=True` lists both in `failure_cases`.

Common wrong answers.

- `[-2.0]`: With lazy=True pandera reports every failing value, not only the first.
- `[1, 2]`: failure_case holds the failing values. The row positions are in the index column.

B5-L1 exercise 3

Hints. (1) A Column argument about repeated values. (2) A schema argument about extra columns. (3) `unique` and `strict`.

Answer.

```
schema = pa.DataFrameSchema(
    {"respondent_id": pa.Column(str, unique=True), "score":
    pa.Column(int)},
    strict=True,
)
```

`unique=True` enforces the grain. `strict=True` fails on any unexpected column.

Common wrong answers.

- g2 = `coerce`: coerce converts dtypes. strict rejects unknown columns.
- g1 = `nullable`: nullable allows nulls. unique rejects duplicates.

B5-L1 exercise 4

Hints. (1) Each column is `pa.Column(dtype, checks..., options)`. (2) Columns reject nulls unless nullable=True. (3) strict and coerce go on the DataFrameSchema, lazy on validate.

Answer.

```
import pandas as pd
import pandera.pandas as pa

SCORES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "nps": pa.Column(int, pa.Check.in_range(0, 10)),
        "comment": pa.Column(str, nullable=True),
    },
    strict=True,
    coerce=True,
)


def validate_scores(df: pd.DataFrame) -> pd.DataFrame:
    """Validate an NPS extract. Grain: one row per respondent.

    Raises:
        pa.errors.SchemaErrors: With every failing check.
    """
    return SCORES.validate(df, lazy=True)
```

The schema names every column, enforces the grain with `unique`, bounds the score, and allows nulls only where they are expected.

B5-L1 exercise 5

Hints. (1) What does lazy=True raise? (2) Is SchemaErrors a subclass of SchemaError? (3) Even if it were, what does the except branch return?

Answer. The bug is on line 12, 13.

Fixed version:

```
1  import pandas as pd
2  import pandera.pandas as pa
3
4  WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
       pa.Check.gt(0))}, strict=True)
5
6
7  def load_weights(path: str) -> pd.DataFrame:
8      """Read and validate weights.
9
10     Raises:
11         pa.errors.SchemaErrors: When any check fails.
12     """
13     return WEIGHTS.validate(pd.read_csv(path), lazy=True)
```

Two faults. `lazy=True` raises `SchemaErrors`, which `except SchemaError` does not catch, so the handler is dead code for value failures. And when it does catch, it returns the unvalidated frame. Let the error raise, so the run stops.

Common wrong answers.

- Line 4: The schema is sound: positive, non-null weights, no extra columns.
- Line 11: Validating with lazy=True is right. What happens when it fails?

B5-L2 pydantic models for config and input records

B5-L2 exercise 1

Hints. (1) What is the default value of the extra setting? (2) Does ignore warn? (3) Change it with model_config.

Answer. b) The key is ignored silently. Because: r1) The default extra setting is "ignore". Set extra="forbid" to reject unknown keys.

Default `extra="ignore"` drops unknown keys. Config models should use `extra="forbid"`.

Common wrong answers.

- `a`: Only with ConfigDict(extra="forbid"). The default ignores extras.
- `c`: pydantic does not guess spellings. minbase is a different key.
- `d`: No warning. That is why extra="forbid" matters for config files.

- `b|r2`: Right answer, wrong reason. It checks keys when you set extra="forbid".

B5-L2 exercise 2

Hints. (1) Is "30" accepted in lax mode? (2) Does model_copy run validators? (3) It does not.

Answer.

```
-1
```

Lax mode converts `"30"` to `30`. `model_copy(update=...)` then sets `-1` without validation. Rebuild with `model_validate` instead.

Common wrong answers.

- `30`: model_copy applies the update. It just does not validate it.
- `ValidationError`: model_copy(update=...) skips validation, so gt=0 is never checked.

B5-L2 exercise 3

Hints. (1) The extra setting that raises. (2) The config key that blocks assignment. (3) Less than or equal.

Answer.

```python
class Response(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    respondent_id: str
    nps: int = Field(ge=0, le=10)
```

`extra="forbid"`, `frozen=True` and `le=10` encode the rules in the model.

Common wrong answers.

- g1 = `ignore`: ignore is the default and drops unknown keys silently.
- g3 = `lt`: lt=10 excludes 10. NPS allows 10, so use le.

B5-L2 exercise 4

Hints. (1) Bounds go in Field(gt=...). (2) Unknown keys and assignment are controlled in model_config. (3) `ConfigDict(extra="forbid", frozen=True)` and `model_validate(raw)`.

Answer.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    """Settings for one weighting run."""
```

```
6
7       model_config = ConfigDict(extra="forbid", frozen=True)
8
9       weight_col: str
10      min_base: int = Field(default=50, gt=0)
11      cap: float = Field(default=5.0, gt=1.0)
12
13
14  def load_config(raw: dict[str, object]) -> WeightingConfig:
15      """Validate a parsed YAML or JSON mapping.
16
17      Raises:
18          pydantic.ValidationError: On any invalid, missing or unknown key.
19      """
20      return WeightingConfig.model_validate(raw)
```

Every rule lives in the model: required fields, bounds, no extras, no changes. `model_validate` reports all failures at once.

B5-L2 exercise 5

Hints. (1) Which pydantic methods validate? (2) Does model_copy? (3) Rebuild with model_validate from model_dump().

Answer. The bug is on line 14.

Fixed version:

```
1  from pydantic import BaseModel, ConfigDict, Field
2
3
4  class Cfg(BaseModel):
5      """Run settings."""
6
7      model_config = ConfigDict(extra="forbid", frozen=True)
8
9      min_base: int = Field(default=50, gt=0)
10
11
12 def with_min_base(cfg: Cfg, value: int) -> Cfg:
13     """Return a validated copy of cfg with a new min_base."""
14     return Cfg.model_validate({**cfg.model_dump(), "min_base": value})
```

`model_copy(update=...)` skips validation. Dump, merge and `model_validate` so the bound applies to the new value.

Common wrong answers.

- Line 9: The bound is right. It is never checked on the copy.
- Line 7: frozen=True is right. model_copy builds a new object, so frozen does not stop it.

B5-L3 Validate at the boundaries, with severity

B5-L3 exercise 1

Hints. (1) Where does data you do not control arrive? (2) Where do you make promises to someone else? (3) In and out.

Answer. b) After reading, and before writing. Because: r1) Those are the boundaries where data changes owner. Tests cover the pure middle.

Validate what you receive and what you deliver. Test the logic in between.

Common wrong answers.

- `a`: Checks inside a tested pure core add noise. The core's inputs are already validated.
- `c`: A bad input would pass through every transform first, and the error would name the wrong step.
- `d`: Unit tests check your code on test data. They say nothing about this week's supplier file.
- `b|r2`: Right answer, wrong reason. The point is ownership boundaries, not speed.

B5-L3 exercise 2

Hints. (1) Is 0.0 truthy? (2) Which branch runs? (3) The fallback.

Answer.

```
1 no data
```

`0.0` is falsy, so the best possible result is reported as missing data. Test with `is not None`.

Common wrong answers.

- `pass`: 0.0 is falsy, so `if share:` skips the comparison. Use `if share is not None:`.

B5-L3 exercise 3

Hints. (1) Check input before using it. (2) Check output before delivering it. (3) Log before raising.

Answer.

1. Read the supplier file
2. Run input checks and collect results
3. Log all results, then raise if any CRITICAL failed
4. Run the pure transforms

5. Run output checks on the result
6. Write the client table

Read, check, enforce, transform, check, write. Each boundary gets its checks before the data moves on.

Common wrong answers.

- Input checks must run before the transforms, so bad input never reaches them.
- Output checks after writing are too late. The client already has the table.

B5-L3 exercise 4

Hints. (1) Compute n once. The share is None when n is 0. (2) Build one QaResult per check. Use `share is not None and share <= max_missing`. (3) enforce filters on severity == "CRITICAL" and not passed.

Answer.

```python
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) -> list[QaResult]:
    """Evaluate input checks. Never raises.

    Args:
        df: Input with a weight column.
        min_rows: Minimum acceptable row count.
        max_missing: Largest acceptable share of null weights.

    Returns:
        One QaResult per check.
    """
    n = len(df)
    share = None if n == 0 else float(df["weight"].isna().sum()) / n
```

```
    return [
        QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
        QaResult("missing_weight", "WARN", share is not None and share <=
    max_missing, share),
    ]


def enforce(results: list[QaResult]) -> None:
    """Stop the run when any CRITICAL check failed.

    Raises:
        RuntimeError: Naming each failed CRITICAL check.
    """
    failed = [r.check for r in results if r.severity == "CRITICAL" and
    not r.passed]
    if failed:
        raise RuntimeError(f"critical checks failed: {failed}")
```

`run_checks` measures and never raises, so every result reaches the log. `enforce` decides, and only CRITICAL failures stop the run.

B5-L3 exercise 5

Hints. (1) Does the failure make every output wrong? (2) Or does it affect one cell, handled by a known rule? (3) Grain and weights are foundations.

Answer.

- Duplicate respondents and broken weights make every figure wrong: CRITICAL.
- A small base is a reporting flag, handled by suppression: WARN.
- A rise in missing age is worth a look but does not invalidate the run: WARN.

CRITICAL for failures that invalidate the whole output. WARN for local issues with a known handling rule.

Common wrong answers.

- grain = `warn`: Duplicates double-count respondents in every total. Nothing downstream is correct.
- weights_sum = `warn`: If weights do not sum to the sample, every weighted figure is off. Stop the run.
- base = `critical`: Small bases are expected in some cells. Flag and suppress them. Do not stop the whole run.